

```
q = QuantumRegister(2, 'q')
c = ClassicalRegister(2, 'c')

circuit = QuantumCircuit(q, c)
circuit.x(q[0]) # Pauli x gate applied to first qubit
circuit.cx(q[0], q[1]) # CNOT applied to both qubits
circuit.measure(q, c) # Qubits states are measured

job = execute(circuit, backend, shots=1)

job_monitor(job)
counts = job.result().get_counts()

print('RESULT: ', counts)
```

August 18, 2019

CNOT GATE TUTORIAL WITH CODE

Interested in learning how to program quantum computers? Then check out our Qiskit textbook **Introduction to Quantum Computing with Qiskit** (<https://quantumcomputinguk.org/shop/introduction-to-quantum-computing-with-qiskit-ebook-1>).

Introduction

This tutorial will introduce the user to the CNOT gate and how to implement it on IBMs quantum devices.

The CNOT gate is a multi-qubit gate that consists of two qubits. The first qubit is known as the control qubit and the second is known as the target qubit. If the control qubit is $|1\rangle$ then it will flip the targets qubit state from $|0\rangle$ to $|1\rangle$ or vice versa.

The CNOT gates operation is described by the following matrix:

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Using matrix multiplication let's explore how the CNOT gate operates on the qubits state. Since this gate operates on two qubits the column vectors will have a row for each possible state:

$$00 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad 01 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad 10 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad 11 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

For our first example let's set the control qubit and target qubit to $|0\rangle$ such that the combined state will be $|00\rangle$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1(1) + 0(0) + 0(0) + 0(0) \\ 0(1) + 1(0) + 0(0) + 0(0) \\ 0(1) + 0(0) + 0(0) + 1(0) \\ 0(1) + 0(0) + 1(0) + 0(0) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

This shows that the target qubits state has remained unchanged as the state is $|00\rangle$ Now let's instead set the control qubit to $|1\rangle$ and the target qubit to $|0\rangle$ such that the combined state will be $|10\rangle$:

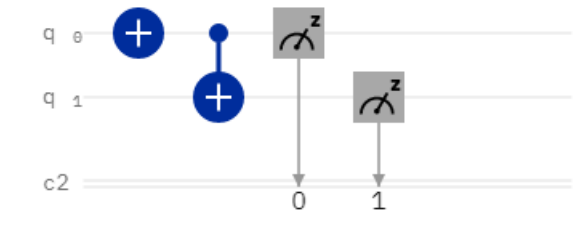
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1(0) + 0(0) + 0(1) + 0(0) \\ 0(0) + 1(0) + 0(1) + 0(0) \\ 0(0) + 0(0) + 0(1) + 1(0) \\ 0(0) + 0(0) + 1(1) + 0(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Which has flipped the target qubit to $|1\rangle$ such that the combined state is $|11\rangle$

Implementation

In Qiskit the CNOT gate can be implemented by initialising two qubits. The first qubits state is flipped to $|1\rangle$ and the second remains $|0\rangle$. Next a CNOT gate is applied where the first qubit is the control and the second is the target qubit. Next both qubits are measured and the results will be sent back from the device.

The full Qiskit code is in the code section below.



Circuit diagram of the program

How to run the program

Copy and paste the code below in to a python file

1. Enter your API token in the IBMQ.enable_account('Insert API token here') part
2. Save and run

Code

```
from qiskit import QuantumRegister, ClassicalRegister,
QuantumCircuit, execute, IBMQ
from qiskit.tools.monitor import job_monitor

IBMQ.enable_account('ENTER API KEY HERE')
provider = IBMQ.get_provider(hub='ibm-q')
backend = provider.get_backend('ibmq_qasm_simulator') # Specifying
qasm simulator as the target device

q = QuantumRegister(2, 'q')
c = ClassicalRegister(2, 'c')

circuit = QuantumCircuit(q, c)
circuit.x(q[0]) # Pauli x gate applied to first qubit
circuit.cx(q[0], q[1]) # CNOT applied to both qubits
circuit.measure(q, c) # Qubits states are measured

job = execute(circuit, backend, shots=1)

job_monitor(job)
counts = job.result().get_counts()

print('RESULT: ', counts)
print('')
print('Press any key to close')
input()
```

Any problems or questions associated with this program? Contact us (<https://quantumcomputinguk.org/contact-us>)

Tagged: 0 (/tutorials/tag/0)

♥ 5 Likes ✕ Share

Newer Post Older Post
Getting real time information on IBM Quantum devices (/tutorials/getting-real-time-information-on-ibm-quantum-devices) Hadamard gate with Code (/tutorials/hadamardgate)

ABOUT US (/ABOUT-US)
CONTACT US (/CONTACT-US)

SCARBOROUGH QUANTUM COMPUTING LTD, 26 PRINCE OF WALES TERRACE, SCARBOROUGH, ENGLAND, YO11 2AN, UNITED KINGDOM

Copyright 2023 Scarborough Quantum Computing Ltd Company Number: 13106408